DEPARTMENT OF SOFTWARE ENGINEERING
Faculty of Computing & Information Technology (FCIT)
University of the Punjab, Lahore – PAKISTAN



MedBill D1 Agile Report

Claim Intake Validation and CMS Preview

BSEF23-51 MedBill

Bachelors of Science in Software Engineering (2023-2027)

By

Hasan Abdullah BSEF23M552

Irfan Saleem BSEF23M553

Supervised by:

Mr Umair Babar
Primary Supervisor

FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY, UNIVERSITY OF THE PUNJAB, LAHORE.

Contents

1. Product Overview....2
2. System Design (High-Level)....2
3. Features & Scope....4
4. Product Backlog....5
5. Prototype Demonstration....6
6. Progress Snapshot....7
7. Definition of Done (DoD)....8
8. Development Approach....9
9. CI/CD Plan (Brief)....9
10. Next Steps (Semester 8)....10

1. Product Overview

Product vision. MedBill will give small clinics and billing teams one system for preparing, checking, tracking and reporting medical billing work. The D1 increment implements the first usable claim-intake workflow: a biller signs in, creates a claim manually or from a PDF, reviews the patient, insurance and service details, runs validation, saves the draft and opens a CMS-1500-style preview.

Problem. Claim details arrive in different formats, and manual re-entry can omit patient identifiers, insurance information, dates, codes or charges. D1 focuses on converting those details into a structured draft that the biller reviews before saving.

Target users. MedBill is intended for small clinics and billing teams. The D1 workflow is demonstrated from the Biller role, while role-based sign-in also distinguishes the Admin role defined in the proposal. The public demonstration uses synthetic data only.

D1 solution. Manual entry and PDF upload lead to the same editable claim, validation result, PostgreSQL record and printable CMS-1500-style preview. In this report, validated means that the D1 presence and format checks passed; it does not mean that a payer accepted the claim.

2. System Design (High-Level)

MedBill is a web application deployed on Vercel. The Next.js interface calls Node.js Route Handlers for authentication, PDF extraction, validation and claim persistence. Prisma stores patient, insurance, claim and source-document metadata in Supabase PostgreSQL. Cloudflare R2 stores private PDFs. PDF.js reads embedded text, and Tesseract.js handles scanned pages before the biller reviews the extracted values.

Next.js browser UI	Next.js Node.js API	Persistent services
Sign in, manual/PDF entry, review, validation, history, CMS preview	Signed sessions, extraction, validation and claim persistence	Prisma/PostgreSQL for records; private R2 for source PDFs

Technology choices

- Frontend: Next.js App Router, React and TypeScript with one guided claim workflow and a printable CMS-1500-style preview.



- Backend: Next.js Node.js Route Handlers for authentication, claim listing and creation, PDF extraction and validation.
- Database: Supabase PostgreSQL through Prisma ORM, with Patient, Insurance, Claim and Document entities.
- Files and extraction: private Cloudflare R2 storage; PDF.js embedded-text extraction and bundled Tesseract.js OCR. Extracted values always require human review.
- Deployment and checks: Vercel production, Prisma migrations, lint, TypeScript build and Playwright browser tests against local and production URLs.

3. Features & Scope

Core Features (Epics)

- **Access and roles:** Biller and Admin demo sign-in with an eight-hour signed session
- **Manual claim intake:** patient, insurance and single-service entry
- **Document intake:** PDF upload with embedded-text extraction or local OCR
- **Human review:** correction of extracted or entered values before validation
- **Claim validation:** explainable required-field, code-format, date and amount checks
- **Persistence and history:** PostgreSQL claim storage and recent-claim history
- **Claim preview:** CMS-1500-style preview with browser print or Save as PDF

D1 implementation boundary

Implemented	Partial or limited	Remaining proposal work
Role-based demo sign-in	Demo accounts; no production identity or user administration	Organization users, password recovery and audit administration
Manual patient and insurance entry	Creates a new patient and policy per claim	Full patient search, edit, deduplication and eligibility
PDF extraction and human review	Label-based extraction; OCR can misread fields	Broader document formats and measured extraction accuracy
Claim validation and persistence	Presence and format checks only	Official code lookup, payer rules and electronic submission
CMS-1500-style printable preview	Not an official clearinghouse transaction	Certified export/submission and acknowledgement handling
Recent-claim history	No payment, denial or reporting lifecycle	Payments, denials, financial reports and measured machine learning

D1 does not submit claims to payers, verify eligibility, post payments, manage denials or reconcile finances. It also does not include a trained denial-risk model. These functions remain in the product backlog for later increments.

4. Product Backlog

The backlog uses MoSCoW priorities. Must stories form the demonstrated D1 increment; Should and Could stories remain planned. A story is marked Done only when its stated acceptance criterion passes.

ID	Priority	User story and acceptance criterion	D1 status
AUTH-01	Must	Story: As a biller or admin, I want secure sign-in so that protected functions are not publicly accessible. Acceptance: valid credentials create a signed session; invalid credentials and unauthenticated requests are rejected.	Done
PAT-01	Must	Story: As a biller, I want to enter patient and insurance details so that a claim has the required member information. Acceptance: missing required fields prevent saving.	Done
CLM-01	Must	Story: As a biller, I want to enter provider, service, diagnosis, procedure and charge details so that I can prepare a draft claim. Acceptance: one complete service line passes the D1 format checks.	Done
PDF-01	Must	Story: As a biller, I want to upload a PDF so that available claim details populate an editable form. Acceptance: valid PDFs produce editable fields and invalid files are rejected.	Done
REV-01	Must	Story: As a biller, I want to review and correct entered or extracted values so that I control the saved claim data. Acceptance: the form remains editable before validation and saving.	Done
VAL-01	Must	Story: As a biller, I want clear validation messages so that I can correct missing or malformed data. Acceptance: failed checks are shown and saving remains blocked until required checks pass.	Done
DB-01	Must	Story: As a biller, I want to save a validated draft so that I can find it in recent-claim history. Acceptance: the saved claim persists in PostgreSQL and appears in history.	Done
CMS-01	Must	Story: As a biller, I want a CMS-1500-style preview so that I can review and print the draft. Acceptance: a saved claim opens a populated preview that can be printed or saved as PDF.	Done
USR-01	Should	Story: As an admin, I want to manage users and reference data so that access and shared billing values remain controlled. Acceptance: authorized admins can create, update and deactivate records.	Planned
PAY-01	Could	Story: As a biller, I want to record payments, adjustments and denials so that claim balances and outcomes remain current. Acceptance: posted transactions update the claim balance and status.	Planned
RPT-01	Could	Story: As a billing lead, I want revenue, aging and denial reports so that I can monitor outstanding and collected amounts. Acceptance: report totals reconcile with stored claims and payments.	Planned
AI-01	Could	Story: As a biller, I want measured denial-risk warnings so that I can review higher-risk claims before submission. Acceptance: the approved model reports documented evaluation results and explainable warnings.	Planned

5. Prototype Demonstration

The deployed prototype is available at https://medbill-steel.vercel.app. The demonstration signs in with the Biller demo account and follows two paths. First, the biller enters synthetic patient, insurance and service data, reviews it, validates it and opens the CMS-1500-style preview. Second, the biller uploads the supplied synthetic claim PDF, reviews and corrects the extracted fields, then repeats the validation and preview steps.

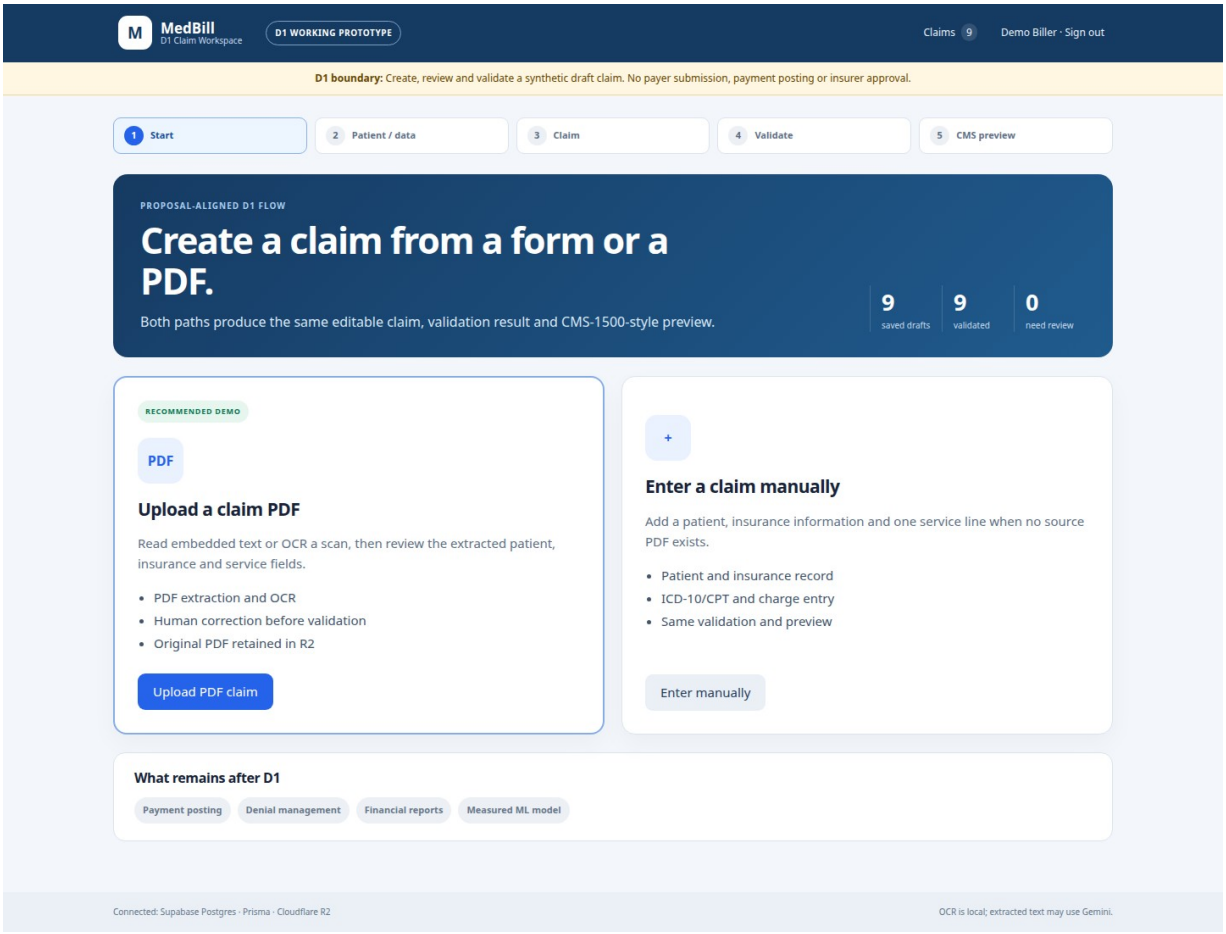


Figure 1 Deployed D1 claim workspace after production verification

Prototype status

- Implemented: role-based demo sign-in, manual and PDF claim intake, human review, validation, PostgreSQL persistence, recent-claim history and a printable CMS-1500-style preview.
- Partially implemented: PDF extraction is label-based and scanned pages use OCR, so the biller must review the result. Validation checks required fields and recognizable formats but does not apply official payer or eligibility rules.
- Not implemented: electronic payer submission, eligibility verification, payment posting, denial management, financial reports and a trained, measured denial-risk model.

6. Progress Snapshot

Live app: https://medbill-steel.vercel.app. GitHub repository: https://github.com/hasanabdullah919/medbill_bsef23-51. The deployed D1 source is recorded at commit 4350ab2. Production verification was completed on 14 October 2026.

Evidence	Result	Meaning
Vercel health endpoint	Passed	Production PostgreSQL and R2 are reachable
Local lint and production build	Passed	ESLint, TypeScript and Next.js compilation succeeded
Local Playwright suite	5 of 5 passed	Authentication, manual entry, PDF extraction, validation, persistence and preview passed
Production Playwright suite	5 of 5 passed	The same complete behavior passed on the live URL
Prisma migration	Applied	Patient, Insurance and Claim tables exist in production PostgreSQL
Protected API access	HTTP 401	Requests without a valid session are rejected

Sprint summary. During this sprint, we refined the backlog, implemented the claim-intake increment, tested it locally and in production, and deployed it. AUTH-01 through CMS-01 are Done. User administration, payments, reporting, payer integrations, measured machine-learning work and hosted CI remain Planned. OCR variability and private document storage were handled through mandatory human review, protected R2 storage and bundled runtime assets.

7. Definition of Done (DoD)

Definition of Ready

A story is Ready for a sprint when the team can implement and verify it without unresolved scope or access questions.

- The story identifies the user, need and benefit and has a MoSCoW priority.
- The acceptance criterion is specific and testable.
- Dependencies, data, design and technical impact are understood.
- Required synthetic data and development access are available, and no known blocker prevents implementation.

Definition of Done

A story is Done only after its stated behavior and acceptance criterion pass with synthetic data. Done applies to that story, not to payer integration or the full product.

- Protected pages and APIs require a valid role-checked session.
- Manual and PDF inputs reach the same review, validation, persistence and preview path.
- Lint, build, migration, local tests and production tests pass.
- The interface and report state the D1 boundary and use no private data or secrets.

8. Development Approach

We use Scrum sprints with planning, review and retrospective activities, as described in the proposal. The D1 increment covers one complete path from claim input to a stored, printable draft. Later sprints will add user administration, payer integration, payments and denials, reporting and measured machine-learning work.

Branching strategy

- main contains the stable, published application and aligned submission documentation.
- testing is the integration branch for work that is not yet ready for main.
- For subsequent stories, the team will create feature/<story-id> or fix/<issue-id> branches before review and merge.

Change workflow

For each story, we confirm readiness, implement the change on an isolated branch, run lint, build and relevant tests, review the result, merge it and verify the deployed behavior. Vercel deployment and production Playwright checks are working; GitHub Actions and protected branch gates remain planned.

Team responsibilities

- Hasan Abdullah serves as Scrum Master and Backend and AI Lead, as assigned in the proposal.
- Irfan Saleem serves as Product Owner and Frontend and Database Lead, as assigned in the proposal.
- Both members participate in backlog refinement, implementation review, testing and sprint review.

9. CI/CD Plan (Brief)

Before each manual deployment, we run the local checks. Vercel then builds the Next.js application, and Playwright tests the full workflow against the deployed URL. GitHub Actions and protected branch gates are planned for a later release.

Stage	Check	Current state or planned gate
Local code	npm run lint; npm run build; npm test	Executed before deployment
Database	Prisma generation and migrations	Applied to production PostgreSQL
Live system	Playwright against production plus health/API checks	Executed after deployment
Hosted CI	GitHub Actions, peer review and protected branches	Planned

10. Next Steps (Semester 8)

Target	Planned outcome
D1 feedback	Review the evaluation feedback and update the prioritized backlog before the next sprint.
Sprint 1	Add full patient/policy search, editing, deduplication and audit events.
Sprint 2	Confirm official CMS-1500 export and payer submission requirements; add code and eligibility services.
Sprint 3	Implement payment, adjustment, denial and outstanding-balance workflows with fixtures.
Sprint 4	Build reconciled reports and an approved, measured denial-risk experiment.
Release	Add CI/review gates, operational identity controls, monitoring and privacy review.

Semester 8 work will extend the demonstrated claim-intake increment in prioritized sprints. Planned work includes user and master-data administration, patient and policy management, official code and eligibility services, payer submission requirements, payments and denials, financial reports, a measured denial-risk experiment, hosted CI and production monitoring.